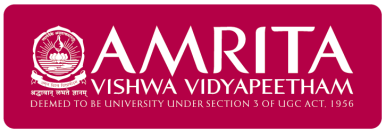


SCHOOL OF
COMPUTING
AMRITAPURI CAMPUS

Course code: 22AIE498 Semester: 7

Course Title: Project Phase 1 Batch: CSE (AIE)

System Architecture Diagram

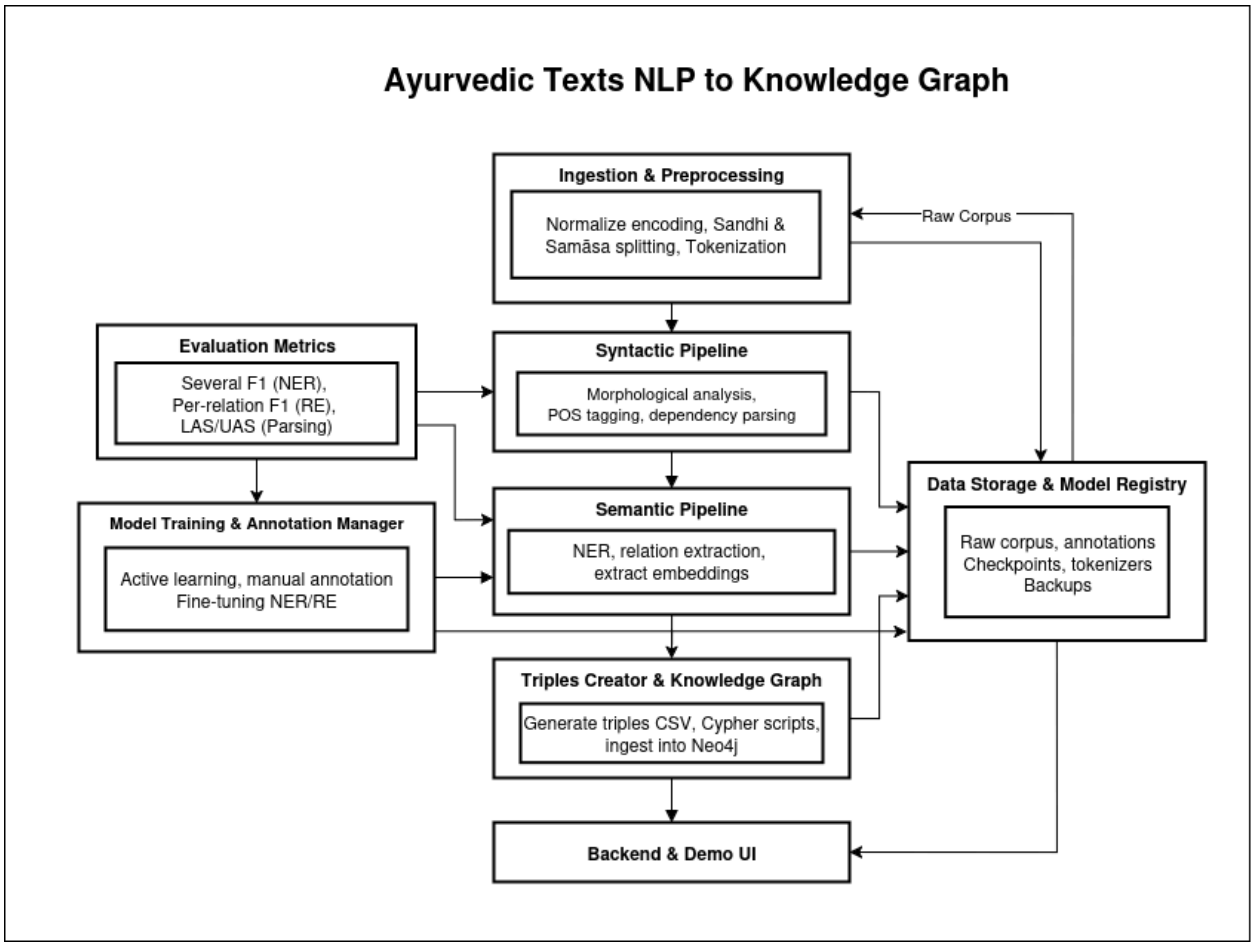
Team Number: 5

Team Members :

Name	Roll Number
Abhiram A S	AM.EN.U4AIE22002
Karthik Narayan C	AM.EN.U4AIE22028
Harigovind C B	AM.EN.U4AIE22119

Project Guide: Jayashree Nair

System Architecture Diagram



1. Ingestion & Preprocessing

This module is responsible for preparing raw Sanskrit Ayurvedic texts (digitized or OCR'd) for downstream analysis. It normalizes encodings (UTF-8), performs sandhi and samāsa splitting, and tokenizes the input sentences. The output is a clean, standardized text represented as tokens and segmentation lattices (e.g., in CoNLLU format), preserving linguistic ambiguity for later disambiguation.

2. Syntactic Pipeline

The syntactic pipeline processes the preprocessed tokens to resolve grammatical structure. It performs morphological analysis, POS tagging, and dependency parsing, enabling the identification of syntactic relations between words. The outputs include POS-tagged tokens and dependency trees, which form the backbone for semantic extraction.

3. Semantic Pipeline (NER + RE + Extract Embeddings)

This module captures the semantic layer of Ayurvedic texts. It uses Named Entity Recognition (NER) to identify entities such as Dravya (herbs), Roga (diseases), Lakṣaṇa (symptoms), and Doṣa (bio-energetic principles). Relation Extraction (RE) links these entities using domain-specific relations (e.g., śamayati, upayujyate). Additionally, embedding models generate semantic representations for tasks like clustering and semantic search. Outputs include BIO-labeled datasets and relation CSVs.

4. Model Training & Annotation Manager

This is the human-in-the-loop component that supports active learning. Annotators manually verify and correct model outputs, creating standard datasets for NER and RE. These datasets are then used to fine-tune models (like Sanskrit-aware Transformers). The module manages the annotation workflow, model training, and evaluation, ensuring iterative improvement of accuracy.

5. Triples Creator & Knowledge Graph Store

The semantic annotations are converted into structured triples of the form (subject, relation, object). These triples are ingested into a Neo4j property graph, where entities form nodes and relations form edges. This module ensures duplicate prevention using MERGE semantics and outputs Cypher scripts, persisted graphs, and visualizations via NetworkX or pyvis.

6. Backend & Demo UI

The backend provides APIs and query endpoints for interacting with the Knowledge Graph. A demo UI (built using Flask) allows users to visualize dependency parses, and explore KG subgraphs. Outputs are presented as JSON responses and interactive graph visualizations, making the system accessible to end users and researchers.

7. Data Storage & Model Registry

This module maintains all persistent resources: raw corpora, annotations, model checkpoints, tokenizers, and KG backups. It ensures reproducibility, version control, and recoverability of the project. By acting as a central repository, it supports both the training process (datasets, models) and the deployment stage (saved graphs, models for inference).